포지셔널 인코딩 구현 및 설명 (Positional Encoding)

목차

- 개요
- 포지셔널 인코딩의 필요성
- 사인/코사인 함수 기반 포지셔널 인코딩
- 수학적 공식 상세 분석
- PyTorch 구현
- 학습 가능한 포지셔널 임베딩과의 비교
- 위치 정보가 어텐션에 미치는 영향
- 시각화 및 분석
- 실습 예제
- 핵심 개념 정리

개요

포지셔널 인코딩(Positional Encoding)은 트랜스포머 아키텍처에서 순서 정보를 모델에 제공하는 핵심 메커니즘입니다. 어텐션 메커니즘은 본질적으로 순서에 무관한(permutation-invariant) 연산이기 때문에, 단어의 위치 정보를 별도로 주입해야 합니다.

포지셔널 인코딩이 해결하는 문제

1. **순서 정보 부재**: 어텐션은 집합(set) 연산과 유사하여 순서 정보가 없음

2. **위치 의존적 의미**: "사과를 먹었다"와 "먹었다 사과를"는 다른 의미

3. **구문 구조**: 문법적 관계는 단어의 상대적 위치에 의존

포지셔널 인코딩의 필요성

어텐션의 순서 무관성

어텐션 메커니즘을 다시 살펴보면:

```
Attention(Q, K, V) = softmax(QK^T / √d_k)V
```

이 연산에서 입력 시퀀스의 순서를 바꿔도 결과가 동일합니다. 예를 들어:

원본: “I love machine learning”

순서 변경: “machine I learning love”

어텐션 계산에서는 두 시퀀스가 동일하게 처리됩니다.

순서 정보의 중요성

자연어에서 순서는 의미를 결정하는 핵심 요소입니다:

1. **문법적 역할**: 주어, 동사, 목적어의 순서

2. **시간적 순서**: “먼저 A를 하고 그 다음 B를 한다”

3. **논리적 관계**: “A이므로 B이다” vs “B이므로 A이다”

따라서 위치 정보를 명시적으로 모델에 제공해야 합니다.## 사인

/코사인 함수 기반 포지셔널 인코딩

트랜스포머에서 사용하는 포지셔널 인코딩은 사인(sine)과 코사인(cosine) 함수를 기반으로 합니다.

기본 공식

위치 `pos` 와 차원 `i` 에 대한 포지셔널 인코딩:

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))

PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

여기서:

- `pos` : 시퀀스에서의 위치 (0, 1, 2, ...)
- `i` : 임베딩 차원의 인덱스 (0, 1, 2, ..., d_model/2-1)
- `d_model` : 모델의 차원 (예: 512)

공식의 직관적 이해

1. **짝수 차원**: 사인 함수 사용
2. **홀수 차원**: 코사인 함수 사용
3. **주파수 변화**: 차원이 증가할수록 주파수가 감소
4. **고유성**: 각 위치마다 고유한 인코딩 벡터 생성

주파수 스펙트럼

각 차원별로 서로 다른 주파수를 사용하는 이유:

```
# 차원별 주파수 계산
def get_frequency(i, d_model):
return 1.0 / (10000 ** (2 * i / d_model))


# 예시: d_model=512일 때
for i in [0, 64, 128, 192, 256]:
```

```
freq = get_frequency(i, 512)

print(f"차원 {i}: 주파수 = {freq:.6f}")
```

출력:

```
차원 0: 주파수 = 1.000000 # 높은 주파수 (빠른 변화)

차원 64: 주파수 = 0.100000

차원 128: 주파수 = 0.010000

차원 192: 주파수 = 0.001000

차원 256: 주파수 = 0.000100 # 낮은 주파수 (느린 변화)
```

이렇게 다양한 주파수를 사용하면:

- **높은 주파수**: 인접한 위치 간의 미세한 차이 포착
- **낮은 주파수**: 먼 거리의 위치 관계 포착##

수학적 공식 상세 분석

1. 주파수 계산

기본 주파수 공식:

```
ω_i = 1 / 10000^(2i/d_model)
```

이를 다시 쓰면:

```
ω_i = 10000^(-2i/d_model) = exp(-2i * ln(10000) / d_model)
```

2. 사인/코사인 함수의 특성

포지셔널 인코딩이 사인/코사인 함수를 사용하는 이유:

주기성 (Periodicity)

```
sin(x + 2π) = sin(x)

cos(x + 2π) = cos(x)
```

상대적 위치 표현

두 위치 간의 관계를 내적으로 표현 가능:

```
PE(pos + k) · PE(pos) = 일정한 값 (k에만 의존)
```

선형 결합 특성

삼각함수의 덧셈 공식을 이용하여:

```
sin(A + B) = sin(A)cos(B) + cos(A)sin(B)

cos(A + B) = cos(A)cos(B) - sin(A)sin(B)
```

이는 상대적 위치 정보를 선형 변환으로 계산할 수 있게 해줍니다.

3. 벡터 표현

위치 `pos` 에 대한 완전한 포지셔널 인코딩 벡터:

```
PE(pos) = [

sin(pos/10000^(0/d_model)), # 차원 0
```

```
cos(pos/10000^(0/d_model)), # 차원 1
sin(pos/10000^(2/d_model)), # 차원 2
cos(pos/10000^(2/d_model)), # 차원 3
...
sin(pos/10000^((d_model-2)/d_model)), # 차원 d_model-2
cos(pos/10000^((d_model-2)/d_model)) # 차원 d_model-1
]
```

4. 상대적 위치 인코딩의 수학적 증명

두 위치 `pos1` 과 `pos2` 사이의 관계:

```
def  relative_position_property():
"""
상대적 위치 특성 증명
"""
import numpy as np
d_model = 512
pos1, pos2 = 5, 8
k = pos2 - pos1 # 상대적 거리
# 포지셔널 인코딩 계산
pe1 = get_positional_encoding(pos1, d_model)
pe2 = get_positional_encoding(pos2, d_model)
# 내적 계산
```

```
dot_product = np.dot(pe1, pe2)

# 이론적 값 (상대적 거리 k에만 의존)

theoretical_value = compute_theoretical_dot_product(k, d_model)

print(f"실제 내적: {dot_product:.6f}")

print(f"이론적 값: {theoretical_value:.6f}")

print(f"차이: {abs(dot_product - theoretical_value):.8f}")

```## PyTorch

구현

기본 포지셔널 인코딩 함수


```python

import torch

import torch.nn as nn

import math

import numpy as np


def  get_positional_encoding(max_seq_len, d_model):

"""

사인/코사인 기반 포지셔널 인코딩 생성

Args:

max_seq_len: 최대 시퀀스 길이

d_model: 모델 차원
```

```
Returns:
pe: 포지셔널 인코딩 텐서 (max_seq_len, d_model)
"""
pe = torch.zeros(max_seq_len, d_model)
# 위치 인덱스 생성 (0, 1, 2, ..., max_seq_len-1)
position = torch.arange(0, max_seq_len).unsqueeze(1).float()
# 차원별 나누기 값 계산
div_term = torch.exp(torch.arange(0, d_model, 2).float() *
-(math.log(10000.0) / d_model))
# 짝수 차원: 사인 함수
pe[:, 0::2] = torch.sin(position * div_term)
# 홀수 차원: 코사인 함수
pe[:, 1::2] = torch.cos(position * div_term)
return pe


# 사용 예제
max_seq_len = 100
d_model = 512


pe = get_positional_encoding(max_seq_len, d_model)
print(f"포지셔널 인코딩 크기: {pe.shape}")
print(f"첫 번째 위치 인코딩: {pe[0, :10]}")
```

```
print(f"두 번째 위치 인코딩: {pe[1, :10]}")
```

포지셔널 인코딩 레이어 클래스

```
class PositionalEncoding(nn.Module):
"""
포지셔널 인코딩 레이어
"""
def __init__(self, d_model, max_seq_len=5000, dropout=0.1):
super(PositionalEncoding, self).__init__()
self.dropout = nn.Dropout(dropout)
# 포지셔널 인코딩 미리 계산
pe = torch.zeros(max_seq_len, d_model)
position = torch.arange(0, max_seq_len).unsqueeze(1).float()
div_term = torch.exp(torch.arange(0, d_model, 2).float() *
-(math.log(10000.0) / d_model))
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
# 배치 차원 추가하여 등록
pe = pe.unsqueeze(0) # (1, max_seq_len, d_model)
self.register_buffer('pe', pe)
def forward(self, x):
"""
```

```
순전파

Args:

x: 입력 임베딩 (batch_size, seq_len, d_model)

Returns:

output: 포지셔널 인코딩이 추가된 임베딩

"""

seq_len = x.size(1)

# 입력 임베딩에 포지셔널 인코딩 추가

x = x + self.pe[:, :seq_len]

return  self.dropout(x)



# 사용 예제

d_model = 512

seq_len = 20

batch_size = 2



pos_encoding = PositionalEncoding(d_model)



# 샘플 입력 임베딩

x = torch.randn(batch_size, seq_len, d_model)

print(f"입력 크기: {x.shape}")



# 포지셔널 인코딩 적용
```

```
output = pos_encoding(x)

print(f"출력 크기: {output.shape}")

```###

단계별 구현 설명


```python

def  detailed_positional_encoding_implementation():

"""

포지셔널 인코딩 구현 과정을 단계별로 설명

"""

max_seq_len = 10

d_model = 8  # 작은 차원으로 시각화

print("=== 포지셔널 인코딩 단계별 구현 ===")

# 1단계: 빈 텐서 생성

pe = torch.zeros(max_seq_len, d_model)

print(f"1단계 - 빈 텐서 생성: {pe.shape}")

# 2단계: 위치 인덱스 생성

position = torch.arange(0, max_seq_len).unsqueeze(1).float()

print(f"2단계 - 위치 인덱스:\n{position.squeeze()}")

# 3단계: 차원별 나누기 값 계산

div_term = torch.exp(torch.arange(0, d_model, 2).float() *

-(math.log(10000.0) / d_model))

print(f"3단계 - 나누기 값: {div_term}")
```

```
# 4단계: 각 위치와 차원별 각도 계산
angles = position * div_term # 브로드캐스팅
print(f"4단계 - 각도 행렬 크기: {angles.shape}")
print(f"각도 행렬 (첫 3행):\n{angles[:3]}")
# 5단계: 사인/코사인 적용
pe[:, 0::2] = torch.sin(angles) # 짝수 차원
pe[:, 1::2] = torch.cos(angles) # 홀수 차원
print(f"5단계 - 최종 포지셔널 인코딩:\n{pe}")
return pe


# 실행
pe_detailed = detailed_positional_encoding_implementation()
```

효율적인 구현 (대용량 처리)

```
class EfficientPositionalEncoding(nn.Module):
"""
메모리 효율적인 포지셔널 인코딩 구현
"""
def  __init__(self, d_model, max_seq_len=10000):
super(EfficientPositionalEncoding, self).__init__()
self.d_model = d_model
# 나누기 값만 미리 계산하여 저장
```

```
div_term = torch.exp(torch.arange(0, d_model, 2).float() *
-(math.log(10000.0) / d_model))
self.register_buffer('div_term', div_term)
def forward(self, x):
"""
동적으로 포지셔널 인코딩 계산
"""
batch_size, seq_len, d_model = x.size()
# 현재 시퀀스 길이에 맞는 포지셔널 인코딩만 계산
position = torch.arange(0, seq_len, device=x.device).unsqueeze(1).float()
pe = torch.zeros(seq_len, d_model, device=x.device)
pe[:, 0::2] = torch.sin(position * self.div_term)
pe[:, 1::2] = torch.cos(position * self.div_term)
# 배치 차원 추가하여 더하기
pe = pe.unsqueeze(0) # (1, seq_len, d_model)
return x + pe


# 메모리 사용량 비교
def compare_memory_usage():
"""
일반 구현과 효율적 구현의 메모리 사용량 비교
"""
d_model = 512
```

```
max_seq_len = 10000

# 일반 구현 (모든 위치 미리 계산)

normal_pe = PositionalEncoding(d_model, max_seq_len)

normal_params = sum(p.numel() for p in normal_pe.parameters() if p.requires_grad

normal_buffers = sum(p.numel() for p in normal_pe.buffers())

# 효율적 구현 (필요한 만큼만 계산)

efficient_pe = EfficientPositionalEncoding(d_model, max_seq_len)

efficient_params = sum(p.numel() for p in efficient_pe.parameters() if p.require

efficient_buffers = sum(p.numel() for p in efficient_pe.buffers())

print(f"일반 구현 - 버퍼 크기: {normal_buffers:,}")

print(f"효율적 구현 - 버퍼 크기: {efficient_buffers:,}")

print(f"메모리 절약: {(normal_buffers - efficient_buffers) / normal_buffers * 100


# 실행

compare_memory_usage()

```## 학습 가능

한 포지셔널 임베딩과의 비교

학습 가능한 포지셔널 임베딩

사인/코사인 함수 대신 학습 가능한 파라미터를 사용하는 방법:


```python
```

```
class LearnablePositionalEmbedding(nn.Module):
"""
학습 가능한 포지셔널 임베딩
"""
def  __init__(self, max_seq_len, d_model, dropout=0.1):
super(LearnablePositionalEmbedding, self).__init__()
# 학습 가능한 포지셔널 임베딩 테이블
self.pos_embedding = nn.Embedding(max_seq_len, d_model)
self.dropout = nn.Dropout(dropout)
# 초기화
nn.init.normal_(self.pos_embedding.weight, std=0.02)
def  forward(self, x):
"""
순전파
Args:
x: 입력 임베딩 (batch_size, seq_len, d_model)
"""
batch_size, seq_len, d_model = x.size()
# 위치 인덱스 생성
positions = torch.arange(0, seq_len, device=x.device)
# 포지셔널 임베딩 조회
pos_emb = self.pos_embedding(positions) # (seq_len, d_model)
# 입력에 추가
```

```
x = x + pos_emb.unsqueeze(0) # 브로드캐스팅
return  self.dropout(x)


# 사용 예제
max_seq_len = 1000
d_model = 512


learnable_pe = LearnablePositionalEmbedding(max_seq_len, d_model)
print(f"학습 가능한 파라미터 수: {sum(p.numel() for p in learnable_pe.parameters())
```

두 방법의 비교 분석

```
def  compare_positional_encodings():
"""
사인/코사인 vs 학습 가능한 포지셔널 인코딩 비교
"""
max_seq_len = 100
d_model = 128
# 두 방법으로 포지셔널 인코딩 생성
sinusoidal_pe = PositionalEncoding(d_model, max_seq_len)
learnable_pe = LearnablePositionalEmbedding(max_seq_len, d_model)
# 샘플 입력
x = torch.randn(1, max_seq_len, d_model)
```

```
# 각각 적용
with torch.no_grad():
sin_output = sinusoidal_pe(x)
learn_output = learnable_pe(x)
# 포지셔널 인코딩 벡터 추출
sin_pe = sin_output - x # 원본 입력을 빼서 포지셔널 인코딩만 추출
learn_pe = learn_output - x
print("=== 포지셔널 인코딩 방법 비교 ===")
print(f"사인/코사인 PE 범위: [{sin_pe.min():.3f}, {sin_pe.max():.3f}]")
print(f"학습 가능한 PE 범위: [{learn_pe.min():.3f}, {learn_pe.max():.3f}]")
# 인접한 위치 간 유사도 계산
sin_similarity = torch.cosine_similarity(sin_pe[0, :-1], sin_pe[0, 1:], dim=1)
learn_similarity = torch.cosine_similarity(learn_pe[0, :-1], learn_pe[0, 1:], di
print(f"사인/코사인 PE 인접 유사도 평균: {sin_similarity.mean():.3f}")
print(f"학습 가능한 PE 인접 유사도 평균: {learn_similarity.mean():.3f}")
return sin_pe, learn_pe


# 실행
sin_pe, learn_pe = compare_positional_encodings()
```

장단점 비교

| 특성 | 사인/코사인 PE | 학습 가능한 PE |

|------|----------|----------|

| **파라미터 수** | 0개 (고정) | max_seq_len × d_model |

| **메모리 사용량** | 낮음 | 높음 |

| **일반화 능력** | 훈련 시보다 긴 시퀀스 처리 가능 | 최대 길이 제한 |

| **표현력** | 수학적으로 정의된 패턴 | 데이터에 맞춰 학습 |

| **해석 가능성** | 높음 (주파수 분석 가능) | 낮음 |

| **수렴 속도** | 빠름 (학습 불필요) | 느림 (학습 필요) |

하이브리드 접근법

두 방법의 장점을 결합한 하이브리드 접근법:

```
class HybridPositionalEncoding(nn.Module):
"""
사인/코사인과 학습 가능한 임베딩을 결합한 하이브리드 방법
"""
def __init__(self, d_model, max_seq_len=5000, dropout=0.1):
super(HybridPositionalEncoding, self).__init__()
# 사인/코사인 포지셔널 인코딩 (고정)
self.sinusoidal_pe = PositionalEncoding(d_model, max_seq_len, dropout=0)
# 학습 가능한 조정 파라미터
self.learnable_adjustment = nn.Parameter(torch.zeros(max_seq_len, d_model))
# 가중치 파라미터
self.alpha = nn.Parameter(torch.tensor(0.5)) # 혼합 비율
self.dropout = nn.Dropout(dropout)
def forward(self, x):
```

```
"""
순전파: 사인/코사인 PE와 학습 가능한 조정을 결합
"""
seq_len = x.size(1)
# 사인/코사인 포지셔널 인코딩
sin_pe = self.sinusoidal_pe.pe[:, :seq_len]
# 학습 가능한 조정
learn_adjustment = self.learnable_adjustment[:seq_len].unsqueeze(0)
# 가중 결합
combined_pe = self.alpha * sin_pe + (1 - self.alpha) * learn_adjustment
# 입력에 추가
x = x + combined_pe
return  self.dropout(x)


# 사용 예제
hybrid_pe = HybridPositionalEncoding(d_model=512)
x = torch.randn(2, 50, 512)
output = hybrid_pe(x)
print(f"하이브리드 PE 출력 크기: {output.shape}")
print(f"혼합 비율 (alpha): {hybrid_pe.alpha.item():.3f}")
```## 위치 정보가 어
텐션에 미치는 영향

포지셔널 인코딩 없는 어텐션

먼저 포지셔널 인코딩이 없을 때의 문제점을 확인해보겠습니다:

```python
def  attention_without_positional_encoding():
"""
포지셔널 인코딩 없는 어텐션의 문제점 시연
"""
from attention_mechanism import scaled_dot_product_attention
# 샘플 문장: "The cat sat on the mat"
tokens = ["The", "cat", "sat", "on", "the", "mat"]
seq_len = len(tokens)
d_model = 64
# 단어 임베딩만 사용 (위치 정보 없음)
# 실제로는 각 단어마다 다른 임베딩이지만, 여기서는 단순화
word_embeddings = torch.randn(1, seq_len, d_model)
# Q, K, V 생성을 위한 선형 변환
W_q = torch.randn(d_model, d_model)
W_k = torch.randn(d_model, d_model)
W_v = torch.randn(d_model, d_model)
Q = torch.matmul(word_embeddings, W_q)
K = torch.matmul(word_embeddings, W_k)
```

```
V = torch.matmul(word_embeddings, W_v)

# 어텐션 계산

output, attention_weights = scaled_dot_product_attention(Q, K, V)

print("=== 포지셔널 인코딩 없는 어텐션 ===")

print("어텐션 가중치:")

print(attention_weights.squeeze(0).detach().numpy())

# 순서를 바꾼 입력으로 테스트

shuffled_indices = [2, 0, 4, 1, 5, 3] # "sat The the cat mat on"

shuffled_embeddings = word_embeddings[:, shuffled_indices, :]

Q_shuffled = torch.matmul(shuffled_embeddings, W_q)

K_shuffled = torch.matmul(shuffled_embeddings, W_k)

V_shuffled = torch.matmul(shuffled_embeddings, W_v)

output_shuffled, attention_shuffled = scaled_dot_product_attention(

Q_shuffled, K_shuffled, V_shuffled)

print("\n순서 변경 후 어텐션 가중치:")

print(attention_shuffled.squeeze(0).detach().numpy())

# 두 결과가 동일한지 확인 (순서만 바뀜)

print(f"\n결과 동일성 확인: {torch.allclose(attention_weights, attention_shuffled)

return attention_weights, attention_shuffled



# 실행

attn_no_pos, attn_shuffled = attention_without_positional_encoding()
```

포지셔널 인코딩이 있는 어텐션

```
def  attention_with_positional_encoding():
"""
포지셔널 인코딩이 있는 어텐션의 효과 시연
"""
tokens = ["The", "cat", "sat", "on", "the", "mat"]
seq_len = len(tokens)
d_model = 64
# 단어 임베딩
word_embeddings = torch.randn(1, seq_len, d_model)
# 포지셔널 인코딩 생성
pos_encoding = PositionalEncoding(d_model)
# 단어 임베딩 + 포지셔널 인코딩
embeddings_with_pos = pos_encoding(word_embeddings)
# Q, K, V 생성
W_q = torch.randn(d_model, d_model)
W_k = torch.randn(d_model, d_model)
W_v = torch.randn(d_model, d_model)
Q = torch.matmul(embeddings_with_pos, W_q)
K = torch.matmul(embeddings_with_pos, W_k)
V = torch.matmul(embeddings_with_pos, W_v)
# 어텐션 계산
```

```
output, attention_weights = scaled_dot_product_attention(Q, K, V)

print("=== 포지셔널 인코딩이 있는 어텐션 ===")

print("어텐션 가중치:")

print(attention_weights.squeeze(0).detach().numpy())

# 순서를 바꾼 입력으로 테스트

shuffled_indices = [2, 0, 4, 1, 5, 3]

shuffled_word_embeddings = word_embeddings[:, shuffled_indices, :]

# 중요: 포지셔널 인코딩은 원래 위치 기준으로 적용

shuffled_embeddings_with_pos = pos_encoding(shuffled_word_embeddings)

Q_shuffled = torch.matmul(shuffled_embeddings_with_pos, W_q)

K_shuffled = torch.matmul(shuffled_embeddings_with_pos, W_k)

V_shuffled = torch.matmul(shuffled_embeddings_with_pos, W_v)

output_shuffled, attention_shuffled = scaled_dot_product_attention(

Q_shuffled, K_shuffled, V_shuffled)

print("\n순서 변경 후 어텐션 가중치:")

print(attention_shuffled.squeeze(0).detach().numpy())

# 두 결과가 다른지 확인

print(f"\n결과 차이 확인: {not torch.allclose(attention_weights, attention_shuffle

return attention_weights, attention_shuffled



# 실행

attn_with_pos, attn_shuffled_pos = attention_with_positional_encoding()
```

상대적 위치의 영향 분석

```
def analyze_relative_position_effects():
"""
상대적 위치가 어텐션에 미치는 영향 분석
"""
seq_len = 10
d_model = 128
# 포지셔널 인코딩 생성
pe = get_positional_encoding(seq_len, d_model)
print("=== 상대적 위치 영향 분석 ===")
# 각 위치 쌍 간의 유사도 계산
similarity_matrix = torch.zeros(seq_len, seq_len)
for i in range(seq_len):
for j in range(seq_len):
# 코사인 유사도 계산
similarity = torch.cosine_similarity(pe[i], pe[j], dim=0)
similarity_matrix[i, j] = similarity
print("위치 간 코사인 유사도 행렬:")
print(similarity_matrix.numpy())
# 거리별 평균 유사도 계산
distance_similarities = {}
for distance in range(seq_len):
```

```
similarities = []

for i in  range(seq_len - distance):

j = i + distance

similarities.append(similarity_matrix[i, j].item())

if similarities:

distance_similarities[distance] = np.mean(similarities)

print("\n거리별 평균 유사도:")

for distance, similarity in distance_similarities.items():

print(f"거리 {distance}: {similarity:.4f}")

return similarity_matrix, distance_similarities



# 실행

sim_matrix, dist_similarities = analyze_relative_position_effects()

```##

시각화 및 분석

포지셔널 인코딩 시각화


```python

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np
```

```python
def visualize_positional_encoding():
"""
포지셔널 인코딩 패턴 시각화
"""
max_seq_len = 100
d_model = 128
# 포지셔널 인코딩 생성
pe = get_positional_encoding(max_seq_len, d_model)
# 1. 전체 포지셔널 인코딩 히트맵
plt.figure(figsize=(15, 8))
plt.subplot(2, 2, 1)
plt.imshow(pe.T, cmap='RdBu', aspect='auto')
plt.title('포지셔널 인코딩 전체 패턴')
plt.xlabel('위치 (Position)')
plt.ylabel('차원 (Dimension)')
plt.colorbar()
# 2. 특정 차원들의 변화 패턴
plt.subplot(2, 2, 2)
positions = np.arange(max_seq_len)
# 서로 다른 주파수의 차원들 선택
dims_to_plot = [0, 16, 32, 64, 96]
colors = ['red', 'blue', 'green', 'orange', 'purple']
for dim, color in zip(dims_to_plot, colors):
```

```
plt.plot(positions, pe[:, dim], color=color, label=f'차원 {dim}')

plt.title('차원별 포지셔널 인코딩 변화')

plt.xlabel('위치')

plt.ylabel('인코딩 값')

plt.legend()

plt.grid(True, alpha=0.3)

# 3. 주파수 스펙트럼

plt.subplot(2, 2, 3)

frequencies = []

for i in  range(0, d_model, 2):

freq = 1.0 / (10000 ** (2 * i / d_model))

frequencies.append(freq)

plt.semilogy(range(0, d_model, 2), frequencies, 'bo-')

plt.title('차원별 주파수')

plt.xlabel('차원')

plt.ylabel('주파수 (로그 스케일)')

plt.grid(True, alpha=0.3)

# 4. 인접 위치 간 유사도

plt.subplot(2, 2, 4)

similarities = []

for i in  range(max_seq_len - 1):

sim = torch.cosine_similarity(pe[i], pe[i+1], dim=0)

similarities.append(sim.item())
```

```
plt.plot(range(max_seq_len - 1), similarities, 'g-')
plt.title('인접 위치 간 코사인 유사도')
plt.xlabel('위치')
plt.ylabel('유사도')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
return pe


# 실행
pe_visualized = visualize_positional_encoding()
```

거리별 유사도 분석

```
def  analyze_distance_similarity():
"""
위치 간 거리에 따른 유사도 변화 분석
"""
max_seq_len = 50
d_model = 256
pe = get_positional_encoding(max_seq_len, d_model)
# 거리별 유사도 계산
max_distance = 20
```

```
distance_similarities = []

for distance in  range(1, max_distance + 1):

similarities = []

for i in  range(max_seq_len - distance):

j = i + distance

sim = torch.cosine_similarity(pe[i], pe[j], dim=0)

similarities.append(sim.item())

avg_similarity = np.mean(similarities)

distance_similarities.append(avg_similarity)

# 시각화

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(range(1, max_distance + 1), distance_similarities, 'bo-')

plt.title('거리별 평균 코사인 유사도')

plt.xlabel('위치 간 거리')

plt.ylabel('평균 코사인 유사도')

plt.grid(True, alpha=0.3)

# 특정 위치에서의 다른 모든 위치와의 유사도

plt.subplot(1, 2, 2)

reference_pos = 10

similarities_from_ref = []

for pos in  range(max_seq_len):

sim = torch.cosine_similarity(pe[reference_pos], pe[pos], dim=0)
```

```
similarities_from_ref.append(sim.item())

plt.plot(range(max_seq_len), similarities_from_ref, 'r-')

plt.axvline(x=reference_pos, color='black', linestyle='--', alpha=0.7)

plt.title(f'위치 {reference_pos}에서 다른 위치들과의 유사도')

plt.xlabel('위치')

plt.ylabel('코사인 유사도')

plt.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()

return distance_similarities


# 실행

dist_sims = analyze_distance_similarity()
```

2D 임베딩 공간에서의 위치 표현

```
def  visualize_position_embedding_space():

"""

포지셔널 인코딩을 2D 공간에 투영하여 시각화

"""

from sklearn.decomposition import PCA

from sklearn.manifold import TSNE

max_seq_len = 50
```

```
d_model = 128

pe = get_positional_encoding(max_seq_len, d_model)

# PCA로 2D 투영

pca = PCA(n_components=2)

pe_pca = pca.fit_transform(pe.numpy())

# t-SNE로 2D 투영

tsne = TSNE(n_components=2, random_state=42)

pe_tsne = tsne.fit_transform(pe.numpy())

# 시각화

plt.figure(figsize=(15, 5))

# PCA 결과

plt.subplot(1, 3, 1)

scatter = plt.scatter(pe_pca[:, 0], pe_pca[:, 1],

c=range(max_seq_len), cmap='viridis', s=50)

plt.colorbar(scatter, label='위치')

plt.title('PCA 투영 (2D)')

plt.xlabel('PC1')

plt.ylabel('PC2')

# 인접한 위치들을 선으로 연결

for i in  range(max_seq_len - 1):

plt.plot([pe_pca[i, 0], pe_pca[i+1, 0]],

[pe_pca[i, 1], pe_pca[i+1, 1]],

'gray', alpha=0.3, linewidth=0.5)
```

```
# t-SNE 결과
plt.subplot(1, 3, 2)
scatter = plt.scatter(pe_tsne[:, 0], pe_tsne[:, 1],
c=range(max_seq_len), cmap='viridis', s=50)
plt.colorbar(scatter, label='위치')
plt.title('t-SNE 투영 (2D)')
plt.xlabel('t-SNE 1')
plt.ylabel('t-SNE 2')
# 원본 고차원 공간에서의 거리 vs 2D 투영 거리 비교
plt.subplot(1, 3, 3)
# 원본 거리 계산
original_distances = []
projected_distances = []
for i in  range(0, max_seq_len, 2):
for j in  range(i+1, max_seq_len, 2):
# 원본 유클리드 거리
orig_dist = torch.norm(pe[i] - pe[j]).item()
original_distances.append(orig_dist)
# PCA 투영 거리
proj_dist = np.linalg.norm(pe_pca[i] - pe_pca[j])
projected_distances.append(proj_dist)
plt.scatter(original_distances, projected_distances, alpha=0.6)
plt.xlabel('원본 고차원 거리')
```

```python
plt.ylabel('PCA 투영 거리')

plt.title('거리 보존 정도')

# 대각선 (완벽한 보존)

min_dist = min(min(original_distances), min(projected_distances))

max_dist = max(max(original_distances), max(projected_distances))

plt.plot([min_dist, max_dist], [min_dist, max_dist], 'r--', alpha=0.7)

plt.tight_layout()

plt.show()

return pe_pca, pe_tsne


# 실행 (sklearn이 설치되어 있는 경우)

try:

pe_pca, pe_tsne = visualize_position_embedding_space()

except ImportError:

print("sklearn이 필요합니다: pip install scikit-learn")
```

```##

실습 예제


### 예제 1: 기본 포지셔널 인코딩 생성


```python

def  basic_positional_encoding_example():

"""
```
```

```
기본적인 포지셔널 인코딩 생성 및 분석
"""
print("=== 기본 포지셔널 인코딩 예제 ===")
# 파라미터 설정
max_seq_len = 10
d_model = 8  # 작은 차원으로 시각화
# 포지셔널 인코딩 생성
pe = get_positional_encoding(max_seq_len, d_model)
print(f"포지셔널 인코딩 크기: {pe.shape}")
print("\n각 위치별 포지셔널 인코딩:")
for pos in  range(min(5, max_seq_len)): # 처음 5개 위치만 출력
print(f"위치 {pos}: {pe[pos].numpy()}")
# 특성 분석
print(f"\n전체 값 범위: [{pe.min():.3f}, {pe.max():.3f}]")
print(f"평균: {pe.mean():.6f}")
print(f"표준편차: {pe.std():.6f}")
# 인접 위치 간 차이
print("\n인접 위치 간 L2 거리:")
for pos in  range(min(4, max_seq_len-1)):
distance = torch.norm(pe[pos+1] - pe[pos]).item()
print(f"위치 {pos} -> {pos+1}: {distance:.4f}")
return pe
```

```
# 실행
pe_basic = basic_positional_encoding_example()
```

예제 2: 포지셔널 인코딩과 단어 임베딩 결합

```
def  combine_word_and_positional_embeddings():
"""
단어 임베딩과 포지셔널 인코딩 결합 예제
"""
print("\n=== 단어 임베딩 + 포지셔널 인코딩 예제 ===")
# 예시 문장
sentence = "The quick brown fox jumps"
tokens = sentence.split()
seq_len = len(tokens)
d_model = 64
print(f"문장: {sentence}")
print(f"토큰: {tokens}")
# 단어 임베딩 시뮬레이션 (실제로는 임베딩 테이블에서 조회)
word_embeddings = torch.randn(seq_len, d_model)
print(f"단어 임베딩 크기: {word_embeddings.shape}")
# 포지셔널 인코딩 생성
pe = get_positional_encoding(seq_len, d_model)
print(f"포지셔널 인코딩 크기: {pe.shape}")
```

```
# 결합
combined_embeddings = word_embeddings + pe
print(f"결합된 임베딩 크기: {combined_embeddings.shape}")
# 각 구성 요소의 기여도 분석
word_norm = torch.norm(word_embeddings, dim=1)
pos_norm = torch.norm(pe, dim=1)
combined_norm = torch.norm(combined_embeddings, dim=1)
print("\n각 토큰별 노름 분석:")
print("토큰\t\t단어임베딩\t포지셔널\t결합")
print("-" * 50)
for i, token in  enumerate(tokens):
print(f"{token:10}\t{word_norm[i]:.3f}\t\t{pos_norm[i]:.3f}\t\t{combined_norm[i]
# 상대적 기여도
pos_contribution = pos_norm / (word_norm + pos_norm)
print(f"\n포지셔널 인코딩 평균 기여도: {pos_contribution.mean():.3f}")
return word_embeddings, pe, combined_embeddings


# 실행
word_emb, pe_emb, combined_emb = combine_word_and_positional_embeddings()
```

예제 3: 다양한 시퀀스 길이에서의 일관성 테스트

```
def  test_sequence_length_consistency():
```

```python
"""
다양한 시퀀스 길이에서 포지셔널 인코딩의 일관성 테스트
"""
print("\n=== 시퀀스 길이 일관성 테스트 ===")
d_model = 128
test_lengths = [10, 20, 50, 100]
# 기준이 되는 긴 시퀀스의 포지셔널 인코딩
reference_pe = get_positional_encoding(max(test_lengths), d_model)
print("시퀀스 길이별 일관성 확인:")
for length in test_lengths:
# 해당 길이의 포지셔널 인코딩 생성
pe = get_positional_encoding(length, d_model)
# 기준과 비교 (처음 length개 위치)
reference_subset = reference_pe[:length]
# 차이 계산
difference = torch.norm(pe - reference_subset)
max_diff = torch.max(torch.abs(pe - reference_subset))
print(f"길이 {length:3d}: L2 차이 = {difference:.8f}, 최대 차이 = {max_diff:.8f}")
# 완전히 동일한지 확인
is_identical = torch.allclose(pe, reference_subset, atol=1e-7)
print(f" 동일성: {is_identical}")
return reference_pe
```

```
# 실행
ref_pe = test_sequence_length_consistency()
```

예제 4: 포지셔널 인코딩의 외삽 능력 테스트

```
def  test_extrapolation_ability():
"""
훈련 시퀀스보다 긴 시퀀스에서의 포지셔널 인코딩 성능 테스트
"""
print("\n=== 외삽 능력 테스트 ===")
d_model = 64
train_length = 50  # 훈련 시 최대 길이
test_length = 100  # 테스트 시 길이
# 훈련 길이의 포지셔널 인코딩
train_pe = get_positional_encoding(train_length, d_model)
# 더 긴 테스트 길이의 포지셔널 인코딩
test_pe = get_positional_encoding(test_length, d_model)
print(f"훈련 길이: {train_length}, 테스트 길이: {test_length}")
# 훈련 범위 내에서의 일관성 확인
train_consistency = torch.allclose(train_pe, test_pe[:train_length])
print(f"훈련 범위 내 일관성: {train_consistency}")
# 외삽 범위에서의 패턴 분석
extrapolated_pe = test_pe[train_length:]
```

```
print(f"외삽 범위 크기: {extrapolated_pe.shape}")

print(f"외삽 범위 값 범위: [{extrapolated_pe.min():.3f}, {extrapolated_pe.max():.3

# 외삽 범위에서의 인접 위치 간 유사도

similarities = []

for i in  range(len(extrapolated_pe) - 1):

sim = torch.cosine_similarity(extrapolated_pe[i], extrapolated_pe[i+1], dim=0)

similarities.append(sim.item())

avg_similarity = np.mean(similarities)

print(f"외삽 범위 인접 유사도 평균: {avg_similarity:.4f}")

# 훈련 범위와 외삽 범위 간의 연결성

boundary_similarity = torch.cosine_similarity(

train_pe[-1], test_pe[train_length], dim=0)

print(f"경계 지점 유사도: {boundary_similarity:.4f}")

return train_pe, test_pe



# 실행

train_pe, test_pe = test_extrapolation_ability()
```

예제 5: 실제 트랜스포머 모델에서의 사용

```
def  transformer_integration_example():

"""

실제 트랜스포머 모델에서 포지셔널 인코딩 사용 예제
```

```
"""

print("\n=== 트랜스포머 통합 예제 ===")

# 모델 파라미터

vocab_size = 1000

d_model = 256

max_seq_len = 100

batch_size = 2

seq_len = 20

# 토큰 임베딩 레이어

token_embedding = nn.Embedding(vocab_size, d_model)

# 포지셔널 인코딩 레이어

pos_encoding = PositionalEncoding(d_model, max_seq_len)

# 샘플 입력 (토큰 인덱스)

input_ids = torch.randint(0, vocab_size, (batch_size, seq_len))

print(f"입력 토큰 ID 크기: {input_ids.shape}")

# 토큰 임베딩

token_emb = token_embedding(input_ids)

print(f"토큰 임베딩 크기: {token_emb.shape}")

# 포지셔널 인코딩 추가

final_embeddings = pos_encoding(token_emb)

print(f"최종 임베딩 크기: {final_embeddings.shape}")

# 각 구성 요소의 통계

print("\n구성 요소별 통계:")
```

```
print(f"토큰 임베딩 - 평균: {token_emb.mean():.4f}, 표준편차: {token_emb.std():.4f}

print(f"최종 임베딩 - 평균: {final_embeddings.mean():.4f}, 표준편차: {final_embeddi

# 포지셔널 인코딩의 영향 분석

pos_effect = final_embeddings - token_emb

print(f"포지셔널 효과 - 평균: {pos_effect.mean():.4f}, 표준편차: {pos_effect.std():

return token_emb, final_embeddings



# 실행

token_emb, final_emb = transformer_integration_example()

```## 핵심

개념 정리

1. 포지셔널 인코딩의 핵심 아이디어

- **순서 정보 주입**: 어텐션의 순서 무관성을 해결

- **고정된 패턴**: 학습이 필요 없는 수학적으로 정의된 패턴

- **상대적 위치 표현**: 절대 위치뿐만 아니라 상대적 관계도 인코딩

- **외삽 가능성**: 훈련 시보다 긴 시퀀스에도 적용 가능

2. 수학적 핵심

- **사인/코사인 함수**: 주기적 특성과 선형 결합 가능성

- **다중 주파수**: 다양한 거리의 위치 관계를 동시에 포착

- **스케일링**: 10000^(2i/d_model)로 주파수 조절

- **차원 분할**: 짝수/홀수 차원에 사인/코사인 각각 적용

3. 구현상 핵심

- **미리 계산**: 고정된 패턴이므로 사전 계산 가능

- **메모리 효율성**: 필요한 길이만큼만 생성하는 동적 방법

- **브로드캐스팅**: 배치 처리를 위한 차원 조작

- **드롭아웃**: 정규화를 위한 선택적 적용

4. 포지셔널 인코딩의 장점

1. **파라미터 효율성**: 학습 가능한 파라미터 없음

2. **일반화 능력**: 다양한 시퀀스 길이에 적용 가능

3. **해석 가능성**: 수학적으로 명확한 의미

4. **안정성**: 훈련 과정에서 변하지 않는 일관된 패턴

5. 한계와 개선 방향

한계:

- 고정된 패턴으로 인한 표현력 제한

- 매우 긴 시퀀스에서의 주파수 해상도 문제

- 특정 태스크에 최적화되지 않은 범용적 패턴

개선 방향:

- 학습 가능한 포지셔널 임베딩과의 결합

- 상대적 위치 인코딩 (Relative Positional Encoding)

- 회전 위치 임베딩 (RoPE, Rotary Position Embedding)

- 적응적 포지셔널 인코딩

6. 실무 적용 고려사항

1. **시퀀스 길이**: 최대 예상 길이를 고려한 설정

2. **메모리 사용량**: 긴 시퀀스에서는 동적 생성 고려

3. **정규화**: 드롭아웃 적용 여부 결정

4. **스케일링**: 단어 임베딩과의 크기 균형

7. 다음 단계

포지셔널 인코딩을 이해했으니, 다음 주제들을 학습할 수 있습니다:

- [멀티헤드 어텐션](06_multihead_attention.md): 여러 어텐션 헤드의 병렬 처리

- [인코더 아키텍처](04_encoder_architecture.md): 포지셔널 인코딩을 포함한 전체 인코더 ㅋ

- [디코더 아키텍처](05_decoder_architecture.md): 디코더에서의 포지셔널 인코딩 활용

이해도 확인 질문:

1. 어텐션 메커니즘이 순서에 무관한 이유는 무엇인가요?
2. 사인/코사인 함수를 사용하는 이유와 장점은 무엇인가요?
3. 차원별로 다른 주파수를 사용하는 이유는 무엇인가요?
4. 포지셔널 인코딩이 상대적 위치 정보를 어떻게 표현하나요?
5. 학습 가능한 포지셔널 임베딩과 비교했을 때의 장단점은 무엇인가요?

실습 과제:

1. 위의 코드를 실행하여 포지셔널 인코딩 패턴을 시각화해보세요.
2. 다른 d_model 값으로 포지셔널 인코딩을 생성하고 패턴 변화를 관찰해보세요.
3. 포지셔널 인코딩이 있는 경우와 없는 경우의 어텐션 가중치를 비교해보세요.
4. 훈련 길이보다 긴 시퀀스에서 포지셔널 인코딩의 외삽 성능을 테스트해보세요.

추가 탐구 주제:

1. RoPE (Rotary Position Embedding)와의 비교
2. 상대적 위치 인코딩 (Relative Positional Encoding)
3. 2D 포지셔널 인코딩 (이미지 트랜스포머용)
4. 학습 가능한 포지셔널 인코딩의 최적화 방법